# Einschränkungen (Constraints)

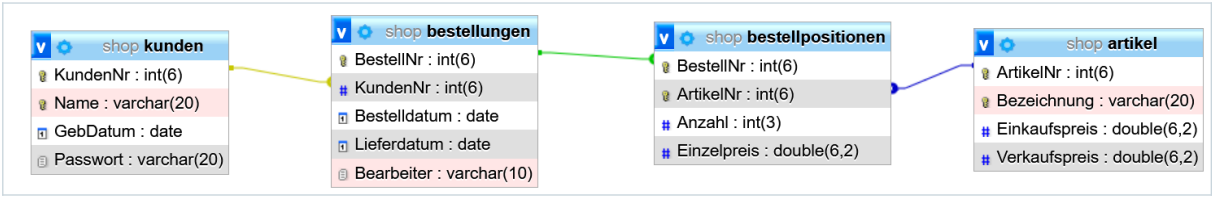
Die meisten DBMS bieten eine Möglichkeit, den zulässigen Wertebereich für eine Spalte durch Constraints (Einschränkung) einzuschränken. Die Regel kann mit CHECK wahlweise direkt bei der Deklaration der Spalte oder als eigener Codeblock nach den Spalten angegeben werden.

In MySQL werden CHECK-Regeln erst ab Version 8.0.16 berücksichtigt, in MariaDB ab Version 10.2.1.

| Constraint | Bedeutung |
|---|---|
| `NOT NULL` | Das Attribut muss bei jedem Datensatz einen Wert enthalten. |
| `CHECK` | Einschränkung des Wertebereichs. |
| `DEFAULT` | Standard-Werte, falls beim Speichern nichts angegeben wird. |
| `UNIQUE` | Die angegebenen Attribute sind eindeutig, es dürfen keine doppelten Werte vorkommen. |

**Beispiel**

```
CREATE TABLE Artikel
(
 ArtikelNr          INT(6)   AUTO_INCREMENT,
 Bezeichnung        VARCHAR(20) NOT NULL UNIQUE DEFAULT 'Elektro',
 Einkaufspreis      DOUBLE(6,2) CHECK (Einkaufspreis >= 0),
 Verkaufspreis      DOUBLE(6,2) CHECK (Verkaufspreis >= 0),
 PRIMARY KEY (ArtikelNr),
 CHECK (Verkaufspreis >= Einkaufspreis)
);
```




## Aufgabe 1: Artikel, Datensätze und Kunden

- Legen Sie die Datenbank `shop` in phpmyadmin mit der Datei `Shop.sql` an.
- Fügen Sie mit Hilfe des Python-Programms `mysql_artikel_einfügen.py` Datensätze in die Tabelle artikel ein.

| | |
|---|---|
| Artikel einfügen mit: | `mysql_artikel_einfügen.py` |
| Datensätze anzeigen mit: | `mysql_anzeigen.py` |
| Kunden einfügen mit: | `kunden_einfügen_input.py` |

Der neu hinzugefügte Kunde hat einen der Artikel bestellt. Speichern Sie die Verknüpfungsinformationen (Fremdschlüssel) in die Zwischentabellen. Arbeiten Sie dazu mit SQL-Variablen und dem insert – Befehl.

## Datensätze einfügen

```
1  import mysql.connector
2
3  try:
4      connection = mysql.connector.connect \
5          (host = "localhost", user = "root", passwd = "")
6  except:
7      print("Keine Verbindung zum Server")
8
9  cursor = connection.cursor()
10
11 cursor.execute("USE Shop")
12 connection.commit()
13
14 sql='''
15 INSERT INTO Artikel VALUES(NULL,"Festplatte",122,209);
16 '''
17 try:
18     cursor.execute(sql)
19     connection.commit()
20 except:
21     print("Fehler")
22
23 cursor.close()
24 connection.close()
```

**1** Treiber / Modul für die Datenbankverbindung importieren: **Zeile 1**

**2** Objekt für die Datenbankverbindung erstellen und die entsprechende Methode parametrieren: **Zeilen 4/5**

**3** Objekt für den Datenbankzugriff (Steuerung der SQL – Befehle) aus der Datenbankverbindung erstellen → über Methoden (hier: `execute()`) SQL an DB übergeben: **Zeilen 9/10/18**

**Fehler mit try und except abfangen**

Falls `connect()` oder `execute()` fehlschlagen, wird ein entsprechender Fehlerhinweis ausgegeben.

## Datensätze abfragen

```
1  import mysql.connector
2
3  try:
4      connection = mysql.connector.connect(host="localhost",
           user="root", passwd="", db="shop")
5  except:
6      print("Keine Verbindung zum Server")
7
8  sql='''
9  SELECT * FROM artikel
10 '''
11 cursor = connection.cursor()
12 cursor.execute(sql)
13 result = cursor.fetchall()
14 cursor.close()
15 connection.close()
16
17
18 print('Anzahl Datensätze: ', len(result))
19 for datensatz in result:
20     for feld in datensatz:
21        print(feld, end=" ")
22     print()
```

**1** Alle Datensätze des Abfrageergebnisses mit `fetchall()` aus dem Cursor-Objekt abholen und in der Variablen `result` speichern: **Zeile 13**

**2** `result` ist eine Liste von Tupeln: ein Tupel pro Datensatz, ein Element pro Feld → Ausgabe mit `len()` und den `for`-Schleifen: **Zeilen 18–22**

# SQL-Injections

SQL-Injection bedeutet: Ein Angreifer schleust über ein Eingabefeld (z. B. Username oder Suchfeld) eigenen SQL-Code ein und verändert damit die Abfrage, die eigentlich vom Programm vorgegeben ist.

Als Ausgangspunkt dienen die drei Python-Programme `login.py`, `anzeige.py` und `eintrag.py`. Dort wird die Benutzereingabe direkt in das SQL-Statement eingesetzt – genau das macht den Angriff möglich.

## Das Grundprinzip: unsicher vs. sicher

Ob ein Programm angreifbar ist, entscheidet sich an einer Stelle – wie das SQL-Statement zusammengebaut wird.

**Unsicher:** Die Eingabe wird direkt in den SQL-Text eingesetzt (f-String / Verkettung). Das Programm kann Befehl und Benutzereingabe nicht mehr unterscheiden.

```
# FALSCH – Eingabe landet direkt im SQL-String
sql = f"SELECT * FROM kunden WHERE Name='{user}' AND Passwort='{pw}'"
db.execute(sql)
```

**Sicher:** Im SQL-Text stehen nur Platzhalter. Die Parameter werden getrennt nachgereicht – die Datenbank behandelt sie immer als Wert, nie als Befehl.

```
# RICHTIG – Platzhalter, Parameter werden nachgereicht
sql = "SELECT * FROM kunden WHERE Name=%s AND Passwort=%s"   # MySQL: %s   SQLite: ?
db.execute(sql, (user, pw))
```

> **Schutz durch Platzhalter**
>
> Der Platzhalter ist die Abhilfe aus dem Skript. Wer ihn benutzt, ist geschützt. Um den Angriff überhaupt zeigen zu können, muss man ihn absichtlich weglassen.

## Drei Angriffsszenarien

Alle drei nutzen dieselbe Schwachstelle: Die Eingabe wird Teil des SQL-Befehls. `#` ist das Kommentarzeichen in MySQL, SQLite nutzt `--`.

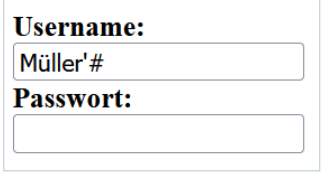
1. Login-Umgehung (Kommentar) – `login.py`

Eingabe im Feld **Username:** `Müller'#`

```
Select * from kunden where Name='Müller'#' and
Passwort='pw'
```

Wirkung: Das `'` schließt den Namen ab, `#` kommentiert die Passwortprüfung weg. Der Login gelingt ohne richtiges Passwort (der Name muss in der Tabelle existieren).

*Eingabe in login.py*



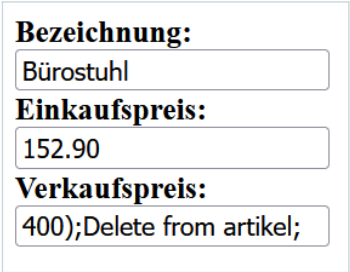
2. Zweites Statement anhängen – `eintrag.py`

Eingaben: **Bezeichnung** `Bürostuhl`, **Einkaufspreis** `152.90`, **Verkaufspreis** `400);Delete from artikel;`

```
insert into artikel values (NULL,'Bürostuhl',
152.90,400);Delete from artikel;);
```

Wirkung: Hinter dem insert wird mit `;` ein zweiter Befehl angehängt – die Tabelle artikel wird geleert. Das funktioniert nur, wenn der Treiber mehrere Statements erlaubt (bei `mysql.connector` mit `multi=True`).

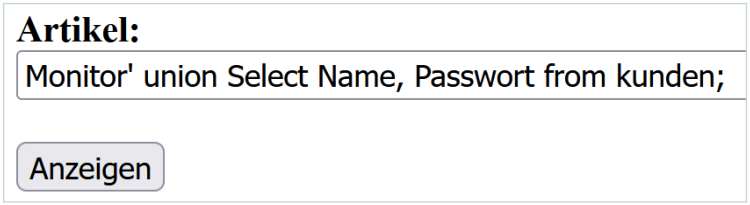
*Eingabe in eintrag.py*




3. UNION – `anzeige.py`

Eingabe im Suchfeld **Artikel:** `Monitor' union Select Name, Passwort from kunden;`

*Eingabe in anzeige.py*




```
Select Bezeichnung, Verkaufspreis from artikel
where Bezeichnung= 'Monitor' union Select Name, Passwort from kunden;'
```

Wirkung: Die Abfrage liefert zusätzlich alle Namen und Passwörter aus der Tabelle `kunden`. Voraussetzung: Beide SELECTs haben gleich viele Felder.

> **Hinweis: # statt ;**
>
> In den Screenshots steht am Ende ein Semikolon. Sauberer ist das Kommentarzeichen `#` am Ende der Eingabe (z. B. `… from kunden #`): Es blendet die restlichen Zeichen aus, die das Programm noch anhängt, und es entsteht kein leeres drittes Statement, das je nach Treiber einen Fehler auslöst.

Führen Sie die UNION-Abfrage zur Kontrolle auch in phpmyadmin (DB: shop) aus.

## Abhilfe: Platzhalter verwenden

- In den SQL-Statements (in Python, PHP etc.) stehen nicht die übergebenen Parameter, sondern Platzhalter.
- Die SQL-Abfrage wird mit den Platzhaltern ausgeführt, die Parameter werden dann nachgereicht –> SQL-Schadcode wird nicht ausgeführt, sondern als String gespeichert.

In Python werden standardmäßig prepared statements bei Datenbankeinträgen benutzt, wenn die Eingaben über Variablen (Platzhalter) erfolgen:

**kunden_einfügen_input.py**

```
16      name = input("Name: ")
17      geb = input("Gebdatum: ")
18      pw = input("Passwort: ")
19
20      cursor.execute("INSERT INTO Kunden VALUES(NULL,%s,%s,%s);",[name,geb,pw])
```

*Keine Ausführung der SQL-Injection*

| KundenNr | Name | GebDatum | Passwort |
|---|---|---|---|
| 8 | Hacker | 1978-12-11 | alles');Delete from kunden; |

# Passwörter in der Datenbank hashen

| KundenNr | Name | GebDatum | Passwort |
|---|---|---|---|
| 1 | Roscher | 1999-11-11 | oma |
| 2 | Meier | 1998-11-11 | 1234 |
| 4 | Werner | 1969-11-11 | $2b$12$bHgbVcUnBym5YhG5J9ep..0/IJWGRZZ5z4CO2WS6.At... |
| 5 | Siemens | 1999-11-11 | $2b$12$guAPgEMRSpH3ji328XkR3u4DBT8QVcUbvdKhjtJ36KE... |

Persönliche Daten, insbesondere Passwörter, sollten verschlüsselt, besser gehasht werden, falls die Datenbank gehackt wird.

| Verfahren | Beschreibung |
|---|---|
| Verschlüsseln | Ein Passwort wird mit einem Verschlüsselungsalgorithmus und einem Schlüssel in einen Geheimtext umgewandelt. Mit dem passenden Schlüssel lässt sich der Geheimtext wieder in den Originaltext entschlüsseln – die Umwandlung ist also umkehrbar. |
| Hashen | Das Passwort wird mit einer Einwegfunktion (Hashfunktion) in einen Hashwert fester Länge umgewandelt. Diese Umwandlung ist nicht umkehrbar: Aus dem Hashwert lässt sich das Passwort nicht zurückrechnen. Beim Login wird die Eingabe erneut gehasht und mit dem gespeicherten Hashwert verglichen. |
| Salt | Damit gleiche Passwörter nicht denselben Hashwert ergeben, wird vor dem Hashen eine Zufallszeichenfolge (Salt) an das Passwort angehängt. Das Salt wird zusammen mit dem Hashwert gespeichert. So erhalten gleiche Passwörter unterschiedliche Hashwerte, und vorberechnete Angriffe (Rainbow Tables) werden erschwert. |

## Aufgabe 2: Passwörter hashen

Führen Sie die beiden Python-Scripte im Ordner `python/Hashen` aus (Modul `bcrypt` ggf. mit `pip install bcrypt` installieren):

- Fügen Sie mit `kunden_pw_hash_einfügen.py` einen neuen Kunden mit Passwort in die Tabelle kunden ein und sehen Sie sich den gespeicherten Hashwert in phpmyadmin an.
- Überprüfen Sie mit `mysql_pw_hash_überprüfen.py` das Passwort dieses Kunden – einmal mit dem richtigen und einmal mit einem falschen Passwort.

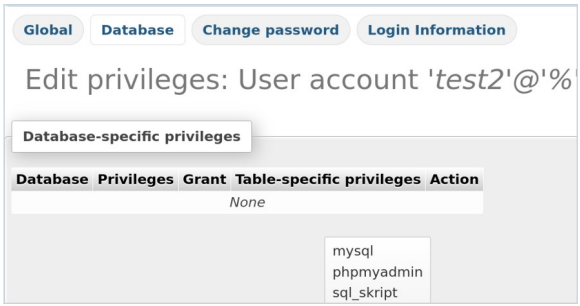
# Rechte auf dem Datenbankserver einschränken

Legen Sie in phpmyadmin über den Reiter **Benutzerkonten** drei Benutzer an:

→ **Benutzerkonto hinzufügen** (noch keine Rechte vergeben).

Notieren Sie sich die Anmeldedaten.

Unter **Benutzerkonten** den Benutzer anklicken und diesem Rechte vergeben:




**globale Rechte** → gelten für alle Datenbanken auf dem Server

**lokale Rechte** → gelten nur auf der ausgewählten Datenbank

## Aufgabe 3: SQL-Statements ausführen

| Benutzer | Rechte |
|---|---|
| Benutzer 1 | bekommt die Rechte SELECT und UPDATE auf die Datenbank sql_skript |
| Benutzer 2 | bekommt die Rechte SELECT und INSERT auf die Datenbank sql_skript |
| Benutzer 3 | bekommt alle Rechte auf alle Datenbanken |

Öffnen Sie drei Terminalfenster und rufen Sie den MySQL-Monitor mit `mysql` auf. Melden Sie sich jeweils mit den neuen Benutzerdaten an und führen Sie die folgenden SQL-Statements bei allen drei Benutzern aus:

> **Hinweis**
>
> Wenn eine Datenbank mit SQL-Befehlen bearbeitet werden soll, muss zuvor mit dem `use`-Befehl die entsprechende Datenbank ausgewählt werden.

- Geben Sie Bezeichnung und Nettopreis der Artikel an, die einen Nettopreis zwischen 200 und 250 € haben.
- Erweitern Sie die Tabelle abteilung um ein neues Feld Umsatz.
- Fügen Sie hier die neue Abteilung Ausbildung ein.
- Ändern Sie den Preis des Artikels ‚ScanJet 2400' auf 79.99 €, lassen Sie sich zur Kontrolle diesen Datensatz anzeigen.

Alle SQL-DML – Statements im Skript `SQL_DML_Befehle.pdf`.

## Aufgabe 4: Daten einfügen und ändern über Python-Scripte

- Erstellen Sie ein Python-Script für Benutzer 2, mit dem er neue Kategorien (neuer Datensatz in der entsprechenden Tabelle) einfügen kann.
- Erstellen Sie ein Python-Script für Benutzer 1, mit dem der Mehrwertsteuersatz aktualisiert werden kann. Für die beiden Mehrwertsteuernummern soll der Wert im Feld Prozent angepasst werden können.

# Datenbank erweitern




Die Tabellen mitarbeiter und kunde besitzen beide die Felder PLZ und Ort. In den Tabellen wird die 3. Normalform verletzt, außerdem treten redundante Daten auf.

Die Datenbank soll deshalb um zwei neue Tabellen erweitert werden: eine Tabelle `ort` und eine Tabelle `plz`. Die Postleitzahl bekommt damit eine eigene Tabelle.

**Warum ein künstliches Primärschlüsselfeld für ort?**

- Ein Ort kann mehrere PLZ haben → die PLZ beschreibt den Ort nicht eindeutig.
- Ortsnamen sind nicht eindeutig („Neustadt“ gibt es mehrfach).
- PLZs können sich ändern; als Primärschlüssel sind sie deshalb ungeeignet.

→ künstliches Primärschlüsselfeld `ort_id`

> **Vorgehensweise**
>
> **Tabelle ort:** `ort_id` PK AUTO_INCREMENT, `ortsname`.
>
> **Tabelle plz:** `plz` PK, `ort_id` FK → ort(ort_id). Ein Ort hat mehrere PLZ (1:n), jede PLZ gehört zu genau einem Ort.
>
> **Reihenfolge:** zuerst ort anlegen, dann plz (wegen des Fremdschlüssels).
>
> **Tabelle mitarbeiter (und kunde):** die Felder PLZ und Ort entfernen und stattdessen `plz` als Fremdschlüssel auf plz(plz) aufnehmen. Der Ort ergibt sich dann über plz → ort.

## Aufgabe 5: Tabellen anlegen und verknüpfen

- Erstellen Sie das SQL-Statement für die Tabelle `ort` (Felder: ort_id PK AUTO_INCREMENT, ortsname) und legen Sie die Tabelle in der DB an.
- Erstellen Sie das SQL-Statement für die Tabelle `plz` (Felder: plz PK, ort_id FK) und legen Sie die Tabelle in der DB an.
- Verknüpfen Sie die neuen Tabellen mit der Tabelle mitarbeiter. Ändern / löschen Sie hier die entsprechenden Felder und erstellen Sie die entsprechenden SQL-Statements.

### Arbeiten mit Variablen

Es können mehrere SQL – Abfragen hintereinander getätigt werden. Die Abfrage nach einem vergebenen Primärschlüsselwert kann in eine Variable `@ps` gespeichert und in der nächsten Abfrage benutzt werden.

**Beispiel**

```
1 BEGIN;
2 SELECT  @ID:= hersteller.HerstellerNr FROM hersteller WHERE hersteller.Name='WD';
3 UPDATE festplatten SET festplatten.hersteller = @ID WHERE festplatten.hersteller ='WD';
4 COMMIT;
```

Bei dieser Transaktion wird der Primärschlüssel eines Herstellers ermittelt und als Sekundärschlüsselwert in die n-Tabelle eingefügt.

# MySQL-Monitor unter Windows

Gehen Sie über die Eingabeaufforderung in den 'bin-Ordner' von MySQL und melden Sie sich auf den MySQL-Server als Administrator mit folgenden Verbindungsparametern an: `mysql -h localhost -u root -p` → Enter; bei der Passwortabfrage Enter.

1. `cd ..` → in den übergeordneten Ordner wechseln
2. `cd xampp` → in den untergeordneten Ordner xampp wechseln

```
C:\>cd xampp

C:\xampp>cd mysql

C:\xampp\mysql>cd bin

C:\xampp\mysql\bin>mysql -h localhost -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with
```

# Benutzerverwaltung MySQL

Jeder Benutzer benötigt ein Konto für den Datenbankserver → Konto: `root@localhost`, mit Hostnamen (oder IP-Adresse) – der Computer, von dem aus das Konto den MySQL-Server erreichen kann – und ein Passwort.

Es können keine Rechte auf eine Datenbank bzw. Tabelle vergeben werden, wenn der Benutzer kein Konto auf dem Datenbankserver besitzt.

Grundsätzlich ist jeder Benutzer, der ein Datenbankobjekt wie z. B. eine Tabelle erstellt, automatisch auch Besitzer dieses Objektes. Alle anderen Benutzer des Systems besitzen für ein Datenbankobjekt eines anderen keine Zugriffsrechte, Ausnahme ist der Administrator.

### Benutzer anlegen

| | |
|---|---|
| `CREATE USER 'jupp'@'%';` | Ein Benutzer mit Benutzernamen 'jupp' wird angelegt.<br>Es ist egal, von welchem PC aus er sich anmeldet. |
| `CREATE USER 'jupp'@'localhost';` | Zugriff nur für den Rechner, auf dem der MySQL-Datenbankserver läuft. |
| `CREATE USER 'willi'@'192.168.2.100';` | Bei Zugriff von einem bestimmten Client. |

### Benutzern Rechte zuordnen

| | |
|---|---|
| `grant all on *.* to 'willi'@'localhost';` | Der Benutzer 'willi' hat alle Rechte auf den MySQL-Datenbankserver. |
| `grant all on hdd.* to 'willi'@'localhost';` | Der Benutzer 'willi' hat nur Rechte auf die Datenbank hdd. |
| `grant select, insert on *.* to 'willi'@'localhost' with grant option;` | Der Benutzer 'willi' darf auf dem Datenbankserver die Funktionen select und insert ausführen und diese Zugriffsrechte anderen Benutzern gewähren. |
| `grant select, update on hdd.Tabelle1 to 'willi'@'localhost';` | Der Benutzer 'willi' darf auf der Tabelle1 der Datenbank hdd nur die Funktionen select und update ausführen. |

**with grant option** – Erlaubt einem Benutzer, anderen Benutzern Zugriffsrechte zu gewähren. Diese Rechte können dem 2. Benutzer vom Administrator nicht entzogen werden.

### Passwörter für Benutzer anlegen

| | |
|---|---|
| `SET PASSWORD FOR 'jupp'@'localhost' = PASSWORD('siemens');` | |

### Widerrufen von Benutzerrechten

| | |
|---|---|
| `REVOKE ALL ON hdd.* FROM 'willi'@'localhost';` | Dem Benutzer 'willi' werden alle Rechte auf die Datenbank hdd entzogen. |
| `REVOKE SELECT, UPDATE ON hdd.* FROM 'willi'@'localhost';` | Der Benutzer 'willi' darf die Funktionen select und update bei der Datenbank hdd nicht einsetzen. |